

Downloads (<http://www.cloudera.com/content/www/en-us/downloads.html>)

Training (<http://www.cloudera.com/content/www/en-us/training.html>)

Support Portal (<http://www.cloudera.com/content/www/en-us/support.html>)

Partners (<http://www.cloudera.com/content/www/en-us/partners.html>)

Developers (<http://www.cloudera.com/content/www/en-us/developers.html>)

Community (<http://community.cloudera.com/>)

cloudera

(<http://www.cloudera.com>)



Cloudera Engineering Blog

(<http://blog.cloudera.com/>)

Best practices, how-tos, use cases, and internals from Cloudera Engineering and the community

The Small Files Problem

February 2, 2009 (<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/>) | By Tom White

(<http://blog.cloudera.com/blog/author/tom/>) | 32 Comments (<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/#comments>)

Categories: (<http://blog.cloudera.com/blog/category/general/>)

(<http://blog.cloudera.com/blog/category/hadoop/>)

Small files are a big problem in Hadoop —or, at least, they are if the number of questions on the user list on [this topic](http://www.google.com/search?hl=en&q=hadoop+%22small+files%22+site%3Anabble.com) (<http://www.google.com/search?hl=en&q=hadoop+%22small+files%22+site%3Anabble.com>) is anything to go by. In this post I'll look at the problem, and examine some common solutions.

Problems with small files and HDFS

A small file is one which is significantly smaller than the HDFS block size (default 64MB). If you're storing small files, then you probably have lots of them (otherwise you wouldn't turn to Hadoop), and the problem is that HDFS can't handle lots of files.

Every file, directory and block in HDFS is represented as an object in the namenode's memory, each of which occupies 150 bytes, as a [rule of thumb](http://www.mail-archive.com/core-user@hadoop.apache.org/msg02835.html) (<http://www.mail-archive.com/core-user@hadoop.apache.org/msg02835.html>). So 10 million files, each using a block, would use about 3 gigabytes of memory. Scaling up much beyond this level is a problem with current hardware. Certainly a billion files is not feasible.

Furthermore, HDFS is not geared up to efficiently accessing small files: it is primarily designed for streaming access of large files. Reading through small files normally causes lots of seeks and lots of hopping from datanode to datanode to retrieve each small file, all of which is an inefficient data access pattern.

Problems with small files and MapReduce

Map tasks usually process a block of input at a time (using the default `FileInputFormat`). If the file is very small and there are a lot of them, then each map task processes very little input, and there are a lot more map tasks, each of which imposes extra bookkeeping overhead. Compare a 1GB file broken into 16 64MB blocks, and

Tweets 

**Cloudera Engineering** 3h
@ClouderaEng

Attn, DC: Cloudera Technology Day is coming on Mar16 -- dig into Kudu, Solr, Security, Hadoop Ops, and Spark j.mp/1QYf5cC

Show Summary

**Cloudera Engineering** 22 Feb
@ClouderaEng

How-to: Integrate Cloudera Director with a Data Pipeline blog.cloudera.com/blog/2016/02/how-to-integrate-cloudera-director-with-a-data-pipeline/

Tweet to @ClouderaEng

Categories

[Accumulo](http://blog.cloudera.com/blog/category/accumulo/) (<http://blog.cloudera.com/blog/category/accumulo/>) (1)
[Avro](http://blog.cloudera.com/blog/category/avro/) (<http://blog.cloudera.com/blog/category/avro/>) (18)
[Bigtop](http://blog.cloudera.com/blog/category/bigtop/) (<http://blog.cloudera.com/blog/category/bigtop/>) (6)

Tags

[analysis](http://blog.cloudera.com/blog/tag/analysis/) (<http://blog.cloudera.com/blog/tag/analysis/>)
[analytics](http://blog.cloudera.com/blog/tag/analytics/) (<http://blog.cloudera.com/blog/tag/analytics/>)
[apache](http://blog.cloudera.com/blog/tag/apache/) (<http://blog.cloudera.com/blog/tag/apache/>)
[apache hadoop](#)

10,000 or so 100KB files. The 10,000 files use one map each, and the job time can be tens or hundreds of times slower than the equivalent one with a single input file.

There are a couple of features to help alleviate the book-keeping overhead: task JVM reuse for running multiple map tasks in one JVM, thereby avoiding some JVM startup overhead (see the `mapred.job.reuse.jvm.num.-tasks` property), and `MultiFileInputSplit` which can run more than one split per map.

Why are small files produced?

There are at least two cases

1. The files are pieces of a larger logical file. Since HDFS has only recently supported appends, a very common pattern for saving unbounded files (e.g. log files) is to write them in chunks into HDFS.
2. The files are inherently small. Imagine a large corpus of images. Each image is a distinct file, and there is no natural way to combine them into one larger file.

These two cases require different solutions. For the first case, where the file is made up of records, the problem may be avoided by calling HDFS's `sync()` method (which came with appends, although see [this discussion](http://www.nabble.com/Hadoop-0.19.1-td21739202.html) (<http://www.nabble.com/Hadoop-0.19.1-td21739202.html>)) every so often to continuously write large files. Alternatively, it's possible to write a program to concatenate the small files together (see Nathan Marz's [post](http://blog.rapleaf.com/dev/?p=36) (<http://blog.rapleaf.com/dev/?p=36>) about a tool called the Consolidator which does exactly this).

For the second case, some kind of container is needed to group the files in some way. Hadoop offers a few options here.

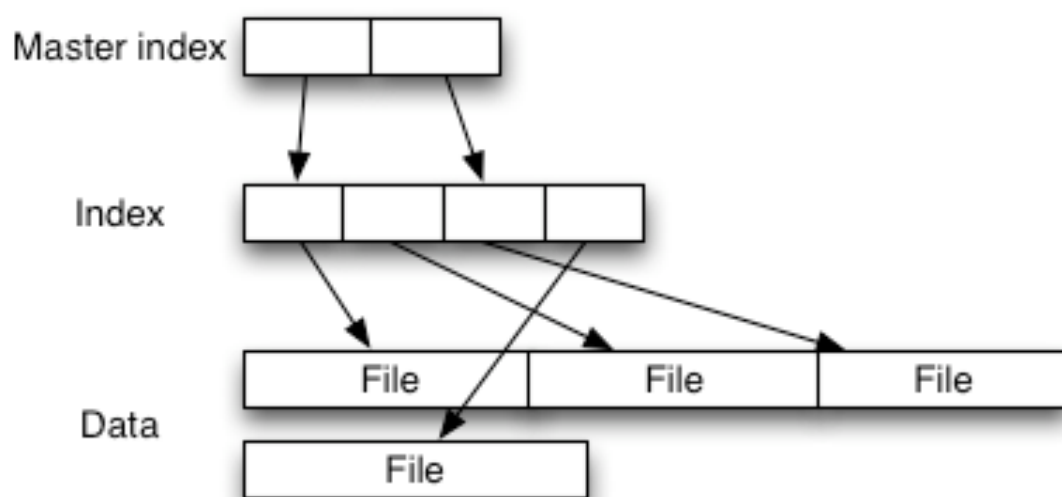
HAR files

[Hadoop Archives](http://hadoop.a-) (<http://hadoop.a->

(<http://blog.cloudera.com/blog/tag/apache-hadoop/>) [Apache HBase](http://blog.cloudera.com/blog/tag/apache-hbase/) (<http://blog.cloudera.com/blog/tag/apache-hbase/>) [apache hive](http://blog.cloudera.com/blog/tag/apache-hive/) (<http://blog.cloudera.com/blog/tag/apache-hive/>) [beta](http://blog.cloudera.com/blog/tag/beta/) (<http://blog.cloudera.com/blog/tag/beta/>) [Big Data](http://blog.cloudera.com/blog/tag/big-data/) (<http://blog.cloudera.com/blog/tag/big-data/>) [CDH](http://blog.cloudera.com/blog/tag/cdh/) (<http://blog.cloudera.com/blog/tag/cdh/>) [cloudera](http://blog.cloudera.com/blog/tag/cloudera/) (<http://blog.cloudera.com/blog/tag/cloudera/>) [Cloudera Manager](http://blog.cloudera.com/blog/tag/cloudera-manager/) (<http://blog.cloudera.com/blog/tag/cloudera-manager/>) [Community configuration](http://blog.cloudera.com/blog/tag/community-configuration/) (<http://blog.cloudera.com/blog/tag/community-configuration/>) [data developer](http://blog.cloudera.com/blog/tag/data-developer/) (<http://blog.cloudera.com/blog/tag/data-developer/>) [developers](http://blog.cloudera.com/blog/tag/developers/) (<http://blog.cloudera.com/blog/tag/developers/>) [development](http://blog.cloudera.com/blog/tag/development/) (<http://blog.cloudera.com/blog/tag/development/>)

pache.org/core/docs/current/hadoop_archives.html) (HAR files) were introduced to HDFS in 0.18.0 to alleviate the problem of lots of files putting pressure on the namenode's memory. HAR files work by building a layered filesystem on top of HDFS. A HAR file is created using the `hadoop archive` command, which runs a MapReduce job to pack the files being archived into a small number of HDFS files. To a client using the HAR filesystem nothing has changed: all of the original files are visible and accessible (albeit using a `har://URL`). However, the number of files in HDFS has been reduced.

HAR File Layout



Reading through files in a HAR is no more efficient than reading through files in HDFS, and in fact may be slower since each HAR file access requires two index file reads as well as the data file read (see diagram). And although HAR files can be used as input to MapReduce, there is no special magic that allows maps to operate over all the files in the HAR co-resident on a HDFS block. It should be possible to build an input format that can take advantage of the improved locality of files in HARs, but it doesn't exist yet. Note that MultiFileInputSplit, even with the improvements in [HADOOP-4565](https://issues.apache.org/jira/browse/HADOOP-4565) (<https://issues.apache.org/jira/browse/HADOOP-4565>) to choose files in a split that are node local, will need a seek per small file. It would be interesting to see the performance of this

ers/) development
[\(http://blog.cloudera.com/blog/tag/development/\)](http://blog.cloudera.com/blog/tag/development/) events
[\(http://blog.cloudera.com/blog/tag/events-2/\)](http://blog.cloudera.com/blog/tag/events-2/) Flume (<http://blog.cloudera.com/blog/tag/flume/>) Hadoop (<http://blog.cloudera.com/blog/tag/hadoop/>)
[hadoop world \(http://blog.cloudera.com/blog/tag/hadoop-world/\)](http://blog.cloudera.com/blog/tag/hadoop-world/) HBase (<http://blog.cloudera.com/blog/tag/hbase/>)
[HDFS \(http://blog.cloudera.com/blog/tag/hdfs/\)](http://blog.cloudera.com/blog/tag/hdfs/) Hive (<http://blog.cloudera.com/blog/tag/hive/>) Hue (<http://blog.cloudera.com/blog/tag/hue/>)
[impala \(http://blog.cloudera.com/blog/tag/impala-2/\)](http://blog.cloudera.com/blog/tag/impala-2/) installation (<http://blog.cloudera.com/blog/tag/installation/>) java (<http://blog.cloudera.com/blog/tag/java/>) log (<http://blog.cloudera.com/blog/tag/log/>) logs (<http://blog.cloudera.com/blog/tag/logs/>)

compared to a SequenceFile, say. At the current time HARs are probably best used purely for archival purposes.

Sequence Files

The usual response to questions about “the small files problem” is: use a SequenceFile. The idea here is that you use the filename as the key and the file contents as the value. This works very well in practice. Going back to the 10,000 100KB files, you can write a program to put them into a single SequenceFile, and then you can process them in a streaming fashion (directly or using MapReduce) operating on the SequenceFile. There are a couple of bonuses too. SequenceFiles are splittable, so MapReduce can break them into chunks and operate on each chunk independently. They support compression as well, unlike HARs. Block compression is the best option in most cases, since it compresses blocks of several records (rather than per record).

It can be slow to convert existing data into SequenceFiles. However, it is perfectly possible to create a collection of SequenceFiles in parallel. (Stuart Sierra has written a very useful [post \(http://stuartsierra.com/2008/04/24/a-million-little-files\)](http://stuartsierra.com/2008/04/24/a-million-little-files) about converting a tar file into a SequenceFile — tools like this are very useful, and it would be good to see more of them). Going forward it’s best to design your data pipeline to write the data at source direct into a SequenceFile, if possible, rather than writing to small files as an intermediate step.

[MapReduce](#)

[\(http://blog.cloudera.com/blog/tag/mapreduce/\)](http://blog.cloudera.com/blog/tag/mapreduce/) [monitoring](#)

[\(http://blog.cloudera.com/blog/tag/monitoring/\)](http://blog.cloudera.com/blog/tag/monitoring/) [open source](#)

[\(http://blog.cloudera.com/blog/tag/open-source/\)](http://blog.cloudera.com/blog/tag/open-source/) [Pig](#)

[\(http://blog.cloudera.com/blog/tag/pig/\)](http://blog.cloudera.com/blog/tag/pig/) [platform](#) [\(http://blog.cloudera.com/blog/tag/platform/\)](http://blog.cloudera.com/blog/tag/platform/) [python](#)

[\(http://blog.cloudera.com/blog/tag/python/\)](http://blog.cloudera.com/blog/tag/python/) [questions](#) [\(http://blog.cloudera.com/blog/tag/questions/\)](http://blog.cloudera.com/blog/tag/questions/) [release](#)

[\(http://blog.cloudera.com/blog/tag/release/\)](http://blog.cloudera.com/blog/tag/release/) [REST](#) [\(http://blog.cloudera.com/blog/tag/rest/\)](http://blog.cloudera.com/blog/tag/rest/) [Search](#) [\(http://blog.cloudera.com/blog/tag/search/\)](http://blog.cloudera.com/blog/tag/search/) [security](#) [\(http://blog.cloudera.com/blog/tag/security/\)](http://blog.cloudera.com/blog/tag/security/) [sql](#) [\(http://blog.cloudera.com/blog/tag/sql/\)](http://blog.cloudera.com/blog/tag/sql/) [Support](#) [\(http://blog.cloudera.com/blog/tag/support-2/\)](http://blog.cloudera.com/blog/tag/support-2/) [Testing](#) [\(http://blog.cloudera.com/blog/tag/testing/\)](http://blog.cloudera.com/blog/tag/testing/)

[\(http://blog.cloudera.com/blog/tag/rest/\)](http://blog.cloudera.com/blog/tag/rest/) [Search](#) [\(http://blog.cloudera.com/blog/tag/search/\)](http://blog.cloudera.com/blog/tag/search/) [security](#) [\(http://blog.cloudera.com/blog/tag/security/\)](http://blog.cloudera.com/blog/tag/security/) [sql](#) [\(http://blog.cloudera.com/blog/tag/sql/\)](http://blog.cloudera.com/blog/tag/sql/) [Support](#) [\(http://blog.cloudera.com/blog/tag/support-2/\)](http://blog.cloudera.com/blog/tag/support-2/) [Testing](#) [\(http://blog.cloudera.com/blog/tag/testing/\)](http://blog.cloudera.com/blog/tag/testing/)

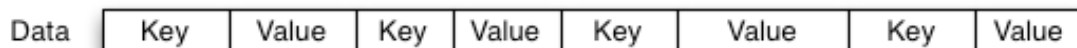
[\(http://blog.cloudera.com/blog/tag/search/\)](http://blog.cloudera.com/blog/tag/search/) [security](#) [\(http://blog.cloudera.com/blog/tag/security/\)](http://blog.cloudera.com/blog/tag/security/) [sql](#) [\(http://blog.cloudera.com/blog/tag/sql/\)](http://blog.cloudera.com/blog/tag/sql/) [Support](#) [\(http://blog.cloudera.com/blog/tag/support-2/\)](http://blog.cloudera.com/blog/tag/support-2/) [Testing](#) [\(http://blog.cloudera.com/blog/tag/testing/\)](http://blog.cloudera.com/blog/tag/testing/)

[\(http://blog.cloudera.com/blog/tag/security/\)](http://blog.cloudera.com/blog/tag/security/) [sql](#) [\(http://blog.cloudera.com/blog/tag/sql/\)](http://blog.cloudera.com/blog/tag/sql/) [Support](#) [\(http://blog.cloudera.com/blog/tag/support-2/\)](http://blog.cloudera.com/blog/tag/support-2/) [Testing](#) [\(http://blog.cloudera.com/blog/tag/testing/\)](http://blog.cloudera.com/blog/tag/testing/)

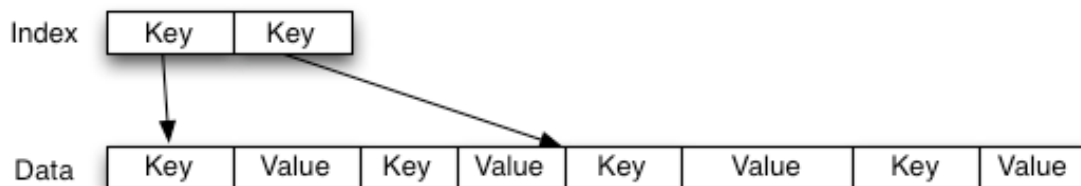
[\(http://blog.cloudera.com/blog/tag/support-2/\)](http://blog.cloudera.com/blog/tag/support-2/) [Testing](#) [\(http://blog.cloudera.com/blog/tag/testing/\)](http://blog.cloudera.com/blog/tag/testing/)

[\(http://blog.cloudera.com/blog/tag/support-2/\)](http://blog.cloudera.com/blog/tag/support-2/) [Testing](#) [\(http://blog.cloudera.com/blog/tag/testing/\)](http://blog.cloudera.com/blog/tag/testing/)

SequenceFile File Layout



MapFile File Layout



Unlike HAR files there is no way to list all the keys in a SequenceFile, short of reading through the whole file. (MapFiles, which are like SequenceFiles with sorted keys, maintain a partial index, so they can't list all their keys either — see diagram.)

SequenceFile is rather Java-centric. [TFile \(https://issues.apache.org/jira/browse/HADOOP-4565\)](https://issues.apache.org/jira/browse/HADOOP-4565) is designed to be cross-platform, and be a replacement for SequenceFile, but it's not available yet.

HBase

If you are producing lots of small files, then, depending on the access pattern, a different type of storage might be more appropriate. [HBase \(http://hadoop.apache.org/hbase/\)](http://hadoop.apache.org/hbase/) stores data in MapFiles (indexed SequenceFiles), and is a good choice if you need to do MapReduce style streaming analyses with the occasional random look up. If latency is an issue, then there are lots of other choices — see Richard Jones' excellent [survey of key-value stores \(http://www.metabrew.com/article/anti-rdbms-a-list-of-distributed-key-value-stores/\)](http://www.metabrew.com/article/anti-rdbms-a-list-of-distributed-key-value-stores/).



Archives

Archives

32 responses on “The Small Files Problem”



Jean-Daniel Cryans

February 2, 2009 at 4:43 pm

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-72>)

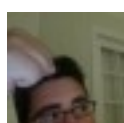
Nice post Tom. Just to add some info with regards to HBase, we are currently integrating TFile in HBase for 0.20 which will hopefully reduce the latency to a reasonable level (looking good). We are also in the process of adding a new cache (HBASE-80).

Pingback: [andy.edmonds.be › links for 2009-02-02](http://andy.edmonds.be/2009/02/links-for-2009-02-02/)

(<http://andy.edmonds.be/2009/02/links-for-2009-02-02/>)

Pingback: [Interesting Finds: 2009.02.10 - gOODi-](http://weblogs.asp.net/yuanjian/archive/2009/02/09/interesting-finds-2009-02-10.aspx)

[DEA.NET \(http://weblogs.asp.net/yuanjian/archive/2009/02/09/interesting-finds-2009-02-10.aspx\)](http://weblogs.asp.net/yuanjian/archive/2009/02/09/interesting-finds-2009-02-10.aspx)



ryan (<http://ryantwopointoh.blogspot.com>)

February 18, 2009 at 3:38 am

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-120>)

Hi Tom,

We are making substantial changes to HBase for 0.20 to improve all manners of performance. Random reads, large scans, etc, etc are all anticipated to be faster.

The new file-format, HFile (hbase-file! get it?) is behind these changes. Check out this for a history of how it came to be:

<http://ryantwopointoh.blogspot.com/2009/02/scalability-what-are-you-doing-about-it.html> (<http://ryantwopointoh.blogspot.com/2009/02/scalability-what-are-you-doing-about-it.html>)



Mark Schnitzius

March 30, 2009 at 5:52 pm

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-600>)

Great article – we are going through many of the same issues. I'm curious to hear how others are dealing with concurrency issues, i.e. external processes adding new data while consolidation of smaller files is taking place.

Pingback: [The Small Files Problem « Ganbatte...!](#)

(<http://wildanm.wordpress.com/2009/04/13/the-small-files-problem/>)

Pingback: [Cloudera Hadoop & Big Data Blog » Blog Archive » What's New in Hadoop Core 0.20](#)
(<http://blog.cloudera.com/blog/2009/05/07/what%e2%80%99s-new-in-hadoop-core-020/>)

Pingback: [Cloudera Hadoop & Big Data Blog » Blog Archive » 10 MapReduce Tips](#) (<http://blog.cloudera.com/blog/2009/05/18/10-mapreduce-tips/>)



Tongjie Chen

May 21, 2009 at 2:58 pm

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-1544>)

Each day we have to deal with around 20,000 gzip files, each gzip file is around 25M(not sure whether 25M falls into *small file* category or not :-)).

Hadoop tasktracker has memory leaking bug prior to 0.19(<http://issues.apache.org/jira/browse/HADOOP-4906> (<http://issues.apache.org/jira/browse/HADOOP-4906>)), which basically disallows us to run job with 20k+ mappers.

We found that MultiFileSplit is very good fit for this problem:

1. convert the whole gzip file to bytes and wrap it with BytesWritable; and wrap multiple BytesWritables into one ArrayWritable
2. Write a CustomMultiFileInputFormat and CustomMultiFileRecordReader where record key is NullWritable, and record value is ArrayWritable.

3 In the map task, for each value(ArrayWritable), We can restore the original bytes from BytesWritable (element of array) and process.

I have used this approach to process around 1.5 millions gzip files (each with 25M in size) in one job and use only 10000 mapper (our cluster size is 61 machines, each 16G memory).

One important thing is that We have to increase the child jvm's -Xmx to big enough to hold ArrayWritable.

Pingback: [pero on anything » Blog Archive » Simulating indexes in Hadoop](#) (<http://pero.blogs.april-mayjune.org/2009/06/06/simulating-indexes-in-hadoop/>)



Konstantin Shvachko

July 13, 2009 at 5:57 pm

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-3360>)

This might be also useful in the context of this discussion.

HADOOP-1687. Name-node memory size estimates...

Name-node object (file or block) size is less than 200 bytes. This is more conservative than 150 bytes, but more practical in my experience.

So the rule of thumb would be: 10 million files require 4 GB of memory.

Pingback: [Hadoop with many small files « Developers blog](#) (<http://www.realdevelopers.com/blog/?p=499>)

Pingback: [GFS Retrospective in ACM Queue — Paper Trail](http://hnr.dnsalias.net/wordpress/2009/08/gfs-retrospective-in-acm-queue/) (<http://hnr.dnsalias.net/wordpress/2009/08/gfs-retrospective-in-acm-queue/>)

Pingback: [hadoop ??????-The Small Files Problem | ?????](http://blog.00rz.com/2010/03/hadoop-%e5%b0%8f%e6%96%87%e4%bb%b6%e7%9a%84%e9%97%ae%e9%a2%98-the-small-files-problem.html) (<http://blog.00rz.com/2010/03/hadoop-%e5%b0%8f%e6%96%87%e4%bb%b6%e7%9a%84%e9%97%ae%e9%a2%98-the-small-files-problem.html>)



Eko Kurniawan Khannedy

April 28, 2010 at 4:25 am

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-11786>)

anyone can tell me, why hadoop default block size is 64MB? why not larger?



Prabhu

July 28, 2010 at 9:06 am

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-12950>)

Great exposition, thanks a lot !

I am faced with a somewhat related problem. Initially I obtained a directory containing a lot of files from the customer. Hadoop was ideal for this, I used the older api for random sampling and totalOrderSort partitioning. Now they are feeding one small file at 6 hour intervals and want it to be processed at the same rate. Now, when I try to run my job, I am getting an ArrayIndexOutOfBoundsException while it tries to write the partition file. I suspect it is because I don't have as many splits as I specify in new RandomSampler(0.1,10,1000). Any help

resolving this would be greatly appreciated.

Thanks very much !

-Prabhu



Prabhu

July 28, 2010 at 11:48 am

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-12954>)

I solved the problem by determining the block size. If it is less than 256 MB, set number of reduce tasks to 1. Else, use random sampler, total order sort partitioner etc.,

Pingback: [Hadoop Archive???????? | ??? | Chocobo](#)

(<http://c.hocobo.net/2010/08/har/>)



Adolf

October 1, 2010 at 3:58 am

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-13958>)

TO solve small files problem in HDFS , DO you know
What can be done?

Pingback: [OCTO talks ! » Scribe : a way to aggregate data and why not, to directly fill the HDFS?](#)

(<http://blog.octo.com/en/scribe-a-way-to-aggregate-data-and-why-not-to-directly-fill-the-hdfs/>)

Pingback: [Hadoop binary files processing introduced by image duplicates finder « eldad levy's playground](http://eldadlevy.wordpress.com/2011/02/05/hadoop-binary-files-processing-introduced-by-image-duplicates-finder/)
(<http://eldadlevy.wordpress.com/2011/02/05/hadoop-binary-files-processing-introduced-by-image-duplicates-finder/>)



Q Ethan (<http://www.exmachinatech.net/go/forqlift/>)

February 10, 2011 at 2:21 pm

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-15436>)

If SequenceFiles are your way to address the small files problem, you may be interested in forqlift:

<http://www.exmachinatech.net/go/forqlift/>
(<http://www.exmachinatech.net/go/forqlift/>)

forqlift is a commandline tool that makes it easy to import/export small files to/from SequenceFiles. (It's also free and open-source.)



Jeans zhong

March 8, 2011 at 10:17 am

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-15692>)

Hii ,Tom, can you explain the following sentence more detailedly ?or give me some reference information?

this sentence:

Reading through small files normally causes lots of seeks and lots of hopping from datanode to datanode to retrieve each small file, all of which is an inefficient data access pattern.

why cause seeks and hopping? I think it only need to

read block of small file at the best datanode.why did you say like that.what 's your reason?
please help me .

Pingback: [Scribe???????? | ????](http://dongxi-cheng.org/mapreduce/scribe-intro/) (<http://dongxi-cheng.org/mapreduce/scribe-intro/>)

Pingback: [HDFS???????? | ????](http://dongxi-cheng.org/mapreduce/hdfs-small-files/) (<http://dongxi-cheng.org/mapreduce/hdfs-small-files/>)

Pingback: [HDFS???????? | ??IT??](http://www.itberry.com/?p=32) (<http://www.itberry.com/?p=32>)

Pingback: [???HDFS???????? » Allen's World](http://allenjoe.net/archives/232) (<http://allenjoe.net/archives/232>)

Pingback: [Summer & cicada~» Blog Archive » Hadoop Archive](http://denqiang.com/?p=213) (<http://denqiang.com/?p=213>)

Pingback: [Scribe日志收集系统介绍 | 稀饭的国度](http://blog.thematice.com/html/y2012/1161_scribe%E6%97%A5%E5%BF%97%E6%94%B6%E9%9B%86%E7%B3%BB%E7%BB%9F%E4%BB%8B%E7%BB%8D.html) (http://blog.thematice.com/html/y2012/1161_scribe%E6%97%A5%E5%BF%97%E6%94%B6%E9%9B%86%E7%B3%BB%E7%BB%9F%E4%BB%8B%E7%BB%8D.html)



TreeMan (<http://weibo.com/treemanliu>)

March 19, 2012 at 4:30 am

(<http://blog.cloudera.com/blog/2009/02/the-small-files->

problem/comment-page-1/#comment-17418)

begin to understand this,a good document !



Viraj Paropkari

November 5, 2012 at 10:42 am

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-22848>)

Hi Tom,

I think there is miscalculation in the sentence.

“So 10 million files, each using a block, would use about 3 gigabytes of memory.”

It should be 1 million instead of 10 million.

Thanks.



Andrew

November 6, 2012 at 2:02 am

(<http://blog.cloudera.com/blog/2009/02/the-small-files-problem/comment-page-1/#comment-23045>)

I have found using SequenceFile.Reader via a direct DF-SClient to be very slow at around 4.5MB/s.

Compared to using FSDataInputStream and reading blocks of bytes directly we get 76MB/s.

Is the SequenceFile reader expected to be this slow?

< HDFS Reliability

Multi-host

(<http://blog.cloudera.com/blog/2009/01/hdfs-reliability/>)

Configuration >

(<http://blog.cloudera.com/blog/2009/01/hdfs-reliability/>)

About Cloudera (<http://www.cloudera.com/content/www/en-us/about-cloudera.html>)

Resources (<http://www.cloudera.com/content/www/en-us/resources.html>)

Contact (<http://www.cloudera.com/content/www/en-us/contact-us.html>)

Careers (<http://www.cloudera.com/content/www/en-us/about-cloudera/careers.html>)

Press Center (<http://www.cloudera.com/content/www/en-us/about-cloudera/press-center.html>)

United States: +1 888 789 1488

International: +1 650 362 0488

(<https://www.linkedin.com/company/cloudera>)

(<https://www.facebook.com/cloudera>)

(<https://twitter.com/cloudera>)

(<http://cloudera.com/contact-us.html>)

Terms & Conditions (<http://www.cloudera.com/content/www/en-us/legal/terms-and-conditions.html>) | **Privacy Policy (<http://www.cloudera.com/content/www/en-us/legal/privacy-policy.html>)**